# Smart medicine storage box

# Final Report

## Team Members

| # | Name | ID |
|---|---|---|
| 1 | Thamer Alharthi | 2035416 |
| 2 | Sari Saeed Alghamdi | 2035130 |

**Team Instructor Name:**

Dr. Waseem Al-zoosh

Table of Content:


| No | Content / Topic | Page NO. |
|---|---|---|
| 1 | Introduction | 3 |
| 2 | Problem Definition | 3 |
| 3 | Circuit Schematic (Proteus) | 3 |
| 4 | Flow Chart | 4 |
| 5 | List of Component | 4 |
| 6 | MPLAB Code | 5-9 |
| 7 | Simulation | 9-10 |
| 8 | Photos of Prototype | 10 |
| 9 | Conclusion | 11 |
| 10 | Reference | 11 |

## Introduction:

Nowadays, most of the products that contain chemicals are negatively affected by heat and humidity, and one of these products is medicines. Most medicines are affected by heat and humidity, and it must be in a suitable and dry environment. In some cases, extreme heat or moisture can damage medicines or change their effectiveness.

So, we are dedicated to finding a device that protects medicines from this harm. The device is a box that contains a display screen for temperature and humidity and is equipped with sensors to warn the user in case there is a significant rise in temperature through a voice alarm and a message that reaches his phone.

## Problem Definition:

Medicines not only lose their effectiveness, but also turn into other very harmful chemicals and we have been thinking about a small box equipped with sensors, gives alarms when the storage area of medicines gets humid or hot to save medicines from damage as if the temperature or humidity exceeds a certain limit, it issues an alarm to warn and operates a fan to reduce the temperature and humidity. We want it to be safe to the user, easy to fix, powered by electricity and costs less than **800 SAR.**

## Circuit Schematic (Proteus):

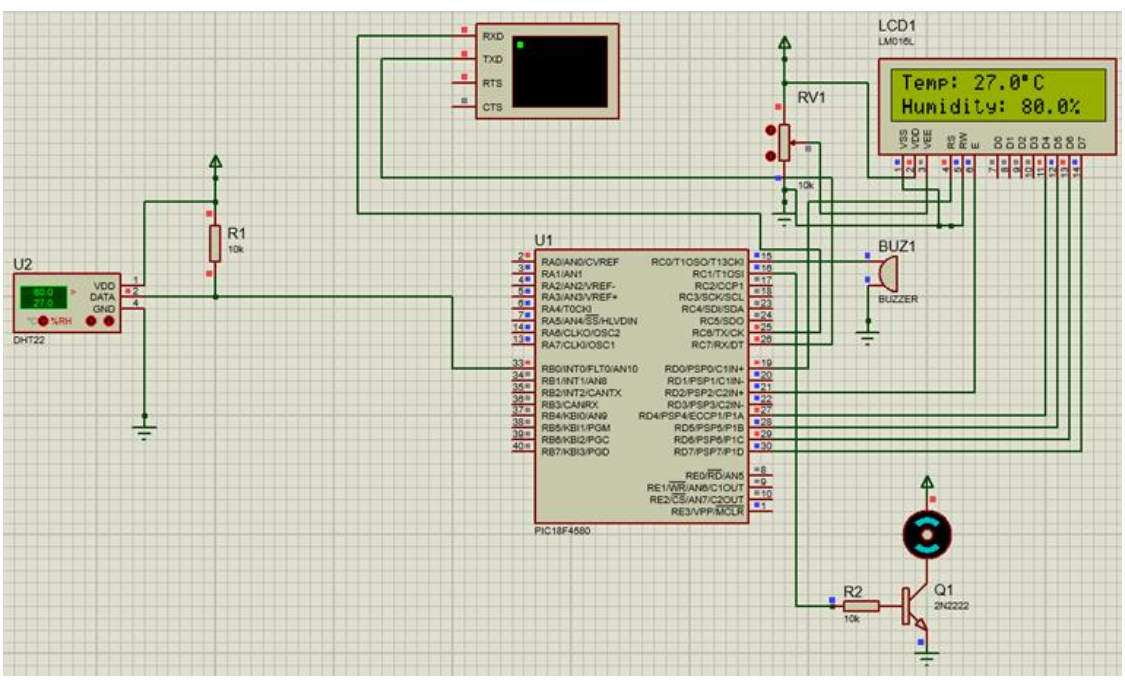


*Figure 1: Circuit diagram (Proteus)*

## Flow Chart:

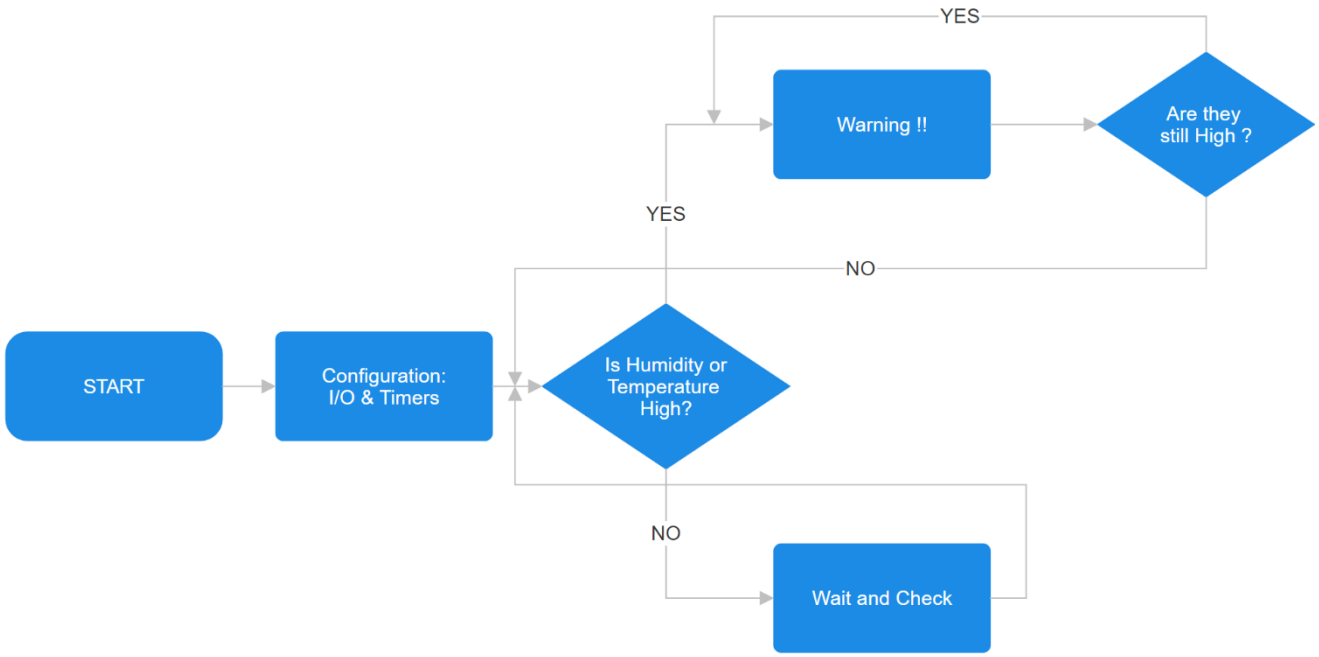


*Figure 2: Flow chart*

## Components & Budget:

| Total | Quantity | Price | Description |
|---|---|---|---|
| 12 | 1 | 12 | Batteries |
| 6 | 1 | 6 | Potentiometer 10k |
| 25 | 1 kit | 25 | Jumper wires |
| 20 | 1 | 20 | Container |
| 57 | 1 | 57 | Humidity and Temperature sensor DHT22 |
| 37 | 1 | 37 | Fan |
| 75 | 1 | 75 | PIC18F4580 |
| 9 | 1 | 9 | buzzer |
| 180 | 1 | 180 | SMS system |
| 15 | 1 | 15 | Breadboard |
| 58 | 1 | 58 | Screen LCD 2*16 |
| 770 SAR | | | |

## MPLAB Code:

This device's C program has been written using this language. The code was transferred to the microcontroller via a PICKIT3. The code is represented by the following:

```c

//File:   main.c

// PIC18F4580 Configuration Bit Settings

// 'C' source line config statements

// CONFIG1H
#pragma config OSC = IRCIO67    // Oscillator Selection bits (Internal oscillator block, port function on RA6 and RA7)
#pragma config FCMEN = OFF      // Fail-Safe Clock Monitor Enable bit (Fail-Safe Clock Monitor disabled)
#pragma config IESO = OFF       // Internal/External Oscillator Switchover bit (Oscillator Switchover mode disabled)

// CONFIG2L
#pragma config PWRT = OFF       // Power-up Timer Enable bit (PWRT disabled)
#pragma config BOREN = OFF      // Brown-out Reset Enable bits (Brown-out Reset disabled in hardware and software)
#pragma config BORV = 3         // Brown-out Reset Voltage bits (VBOR set to 2.1V)

// CONFIG2H
#pragma config WDT = OFF        // Watchdog Timer Enable bit (WDT disabled (control is placed on the SWDTEN bit))
#pragma config WDTPS = 32768    // Watchdog Timer Postscale Select bits (1:32768)

// CONFIG3H
#pragma config PBADEN = OFF     // PORTB A/D Enable bit (PORTB<4:0> pins are configured as digital I/O on Reset)
#pragma config LPT1OSC = OFF    // Low-Power Timer 1 Oscillator Enable bit (Timer1 configured for higher power operation)
#pragma config MCLRE = OFF      // MCLR Pin Enable bit (RE3 input pin enabled; MCLR disabled)

// CONFIG4L
#pragma config STVREN = ON      // Stack Full/Underflow Reset Enable bit (Stack full/underflow will cause Reset)
#pragma config LVP = ON         // Single-Supply ICSP Enable bit (Single-Supply ICSP enabled)
#pragma config BBSIZ = 1024     // Boot Block Size Select bit (1K words (2K bytes) boot block)
#pragma config XINST = OFF      // Extended Instruction Set Enable bit (Instruction set extension and Indexed Addressing mode disabled (Legacy mode))

// CONFIG5L
#pragma config CP0 = OFF        // Code Protection bit (Block 0 (000800-001FFFh) not code-protected)
#pragma config CP1 = OFF        // Code Protection bit (Block 1 (002000-003FFFh) not code-protected)
#pragma config CP2 = OFF        // Code Protection bit (Block 2 (004000-005FFFh) not code-protected)
#pragma config CP3 = OFF        // Code Protection bit (Block 3 (006000-007FFFh) not code-protected)

// CONFIG5H
#pragma config CPB = OFF        // Boot Block Code Protection bit (Boot block (000000-0007FFh) not code-protected)
#pragma config CPD = OFF        // Data EEPROM Code Protection bit (Data EEPROM not code-protected)

// CONFIG6L
#pragma config WRT0 = OFF       // Write Protection bit (Block 0 (000800-001FFFh) not write-protected)
#pragma config WRT1 = OFF       // Write Protection bit (Block 1 (002000-003FFFh) not write-protected)
#pragma config WRT2 = OFF       // Write Protection bit (Block 2 (004000-005FFFh) not write-protected)
#pragma config WRT3 = OFF       // Write Protection bit (Block 3 (006000-007FFFh) not write-protected)

// CONFIG6H
#pragma config WRTC = OFF       // Configuration Register Write Protection bit (Configuration registers (300000-3000FFh) not write-protected)
#pragma config WRTB = OFF       // Boot Block Write Protection bit (Boot block (000000-0007FFh) not write-protected)
#pragma config WRTD = OFF       // Data EEPROM Write Protection bit (Data EEPROM not write-protected)

// CONFIG7L
#pragma config EBTR0 = OFF      // Table Read Protection bit (Block 0 (000800-001FFFh) not protected from table reads executed in other blocks)
#pragma config EBTR1 = OFF      // Table Read Protection bit (Block 1 (002000-003FFFh) not protected from table reads executed in other blocks)
#pragma config EBTR2 = OFF      // Table Read Protection bit (Block 2 (004000-005FFFh) not protected from table reads executed in other blocks)
#pragma config EBTR3 = OFF      // Table Read Protection bit (Block 3 (006000-007FFFh) not protected from table reads executed in other blocks)

// CONFIG7H
#pragma config EBTRB = OFF      // Boot Block Table Read Protection bit (Boot block (000000-0007FFh) not protected from table reads executed in other blocks
```

```c

// #pragma config statements should precede project file includes.
// Use project enums instead of #define for ON and OFF

#include <xc.h>
#include <stdio.h>
#include <string.h>

#include "LCD.h"

#define FREQ 8000000

#define DHT22_PIN PORTBbits.RB0
#define DHT22_PIN_DIR TRISBbits.RB0

#define baud 9600
#define spbrg_value (((FREQ/64)/baud)-1)

int DHT22_ReadData(float *temperature, float *humidity);

void tx_data(unsigned char);
unsigned char rx_data();
void gsm_cmd(unsigned char *);
//void gsm_send_sms();
void gsm_send_sms(const char* phoneNumber, const char* message);

uint16_t delay();

uint16_t del = 0;
uint16_t count = 0;

char error = 0;


__bit sms, sms_state = 0;

void main(void) {
    TRISA = 0x00;
    TRISD = 0x00;
    TRISCbits.RC0 = 0;
    TRISCbits.RC1 = 0;

    OSCCONbits.IRCF = 0x07;
    OSCTUNEbits.TUN = 0x0F;
    OSCCON = 0x73;

    LATAbits.LA0 = 1;
    LATCbits.LATC0 = 0;
    LATCbits.LATC1 = 0;

    float temperature, humidity;
    char lcd_buffer_temp[16];
    char lcd_buffer_humid[16];

    LCD_Init();     /* initialize LCD16x2 */
    sms = 1;
    ADCON1=0x0F;    /* this makes all pins as a digital I/O pins */

    SPBRG=spbrg_value;          // Fill SPBRG register to set the baud rate
    RCSTAbits.SPEN=1;               // To activate serial port (Tx and Rx pins)
    TXSTAbits.TXEN=1;               // Activate Transmissiom
    RCSTAbits.CREN=1;               // Activate Reception
    PIE1bits.RCIE=1;                // Enable Reception interrupt
    INTCONbits.GIE=1;               // Enable Global interrupt
    INTCONbits.PEIE=1;              // Enable Peripheral interrupt
```

```c

    LCD_Clear();
    __delay_ms(500);

    while(1) {

        if(DHT22_ReadData(&temperature, &humidity) == 0) {
            sprintf(lcd_buffer_temp, "Temp: %.1f\337C", temperature);
            sprintf(lcd_buffer_humid, "Humidity: %.1f%%", humidity);
            __delay_ms(1000);
        }
         if(temperature < 27.0 &&  humidity <65.0) {
             LCD_Clear();
                LCD_String_xy(0,0,"Temp.&Hum. are");
                LCD_String_xy(1,0,"in normal limits");
                __delay_ms(1000);


         }

        if(temperature > 27.0 || humidity > 65.0) {
            error = 1;
            LATCbits.LATC0 = 1;
            LATCbits.LATC1 = 1;
            LCD_Clear();
        }
        else {
            error = 0;
            LATCbits.LATC0 = 0;
            LATCbits.LATC1 = 0;
            LCD_Clear();
        }
//
         }
//
         if(error) {
             if((temperature > 27.0 &&  humidity < 65.0)) {
                 LCD_String_xy(0,4,"TOO WARM!");
                 LCD_String_xy(1,4,"Cool Down");

                 if(sms) {
                     gsm_send_sms("+966598720577", "TOO WARM! Cool Down");
                     sms = 0;
                 }
             }

             if((temperature < 27.0 &&  humidity > 65.0)) {
                 LCD_String_xy(0,3,"TOO Humid!!!");

                 if(sms) {
                     gsm_send_sms("+966598720577", "TOO Humid!!!");
                     sms = 0;
                 }
             }

             if((temperature > 27.0) && (humidity > 65.0)) {
                 LCD_String_xy(0,3,"High Temp.");
                 LCD_String_xy(1,3,"High Humid.");

                 if(sms) {
                     gsm_send_sms("+966598720577", "High Temp. and High Humid.!!!");
                     sms = 0;
                 }
             }
         }
         else {
```

```c
            LCD_String_xy(0, 0, lcd_buffer_temp);
            LCD_String_xy(1, 0, lcd_buffer_humid);
            sms = 1;


        }
//
    }
    return;
}

uint16_t delay() {

    del++;
    __delay_ms(1);

    return del;
}

int DHT22_ReadData(float *temperature, float *humidity) {
    unsigned char data[5];
    unsigned char i, j;

    // DHT22 start signal
    DHT22_PIN_DIR = 0;
    LATB0 = 0;    // Pull the data line low
    __delay_ms(20);  // Hold low for at least 18ms (datasheet requirement)
    LATB0 = 1;    // Release the data line
    __delay_us(30);  // Wait for DHT22 response
    DHT22_PIN_DIR = 1;

    // Check for DHT22 response
    if (DHT22_PIN == 0) {
        __delay_us(80);  // Wait for DHT22 to pull the line low
```

```c
        if (DHT22_PIN == 1) {
            __delay_us(80);  // Wait for DHT22 to pull the line high

            // Read data from DHT22
            for (j = 0; j < 5; j++) {
                data[j] = 0;
                for (i = 0; i < 8; i++) {
                    while (DHT22_PIN == 0);  // Wait for the bit to be pulled high
                    __delay_us(30);  // Wait for the duration of the bit
                    if (DHT22_PIN == 1) {
                        data[j] |= (1 << (7 - i));  // Store the bit
                        while (DHT22_PIN == 1);  // Wait for the bit to be pulled low
                    }
                }
            }

            // Verify checksum
            if (data[4] == ((data[0] + data[1] + data[2] + data[3]) & 0xFF)) {
                    // Humidity Calculation
                 *humidity = ((float)data[0] * 256 + data[1]) / 10.0;
                   // Temperature Calculation
                 *temperature = ((float)((data[2] & 0x7F) * 256) + data[3]) / 10.0;

                if (data[2] & 0x80) {
                    *temperature = -*temperature;  // Check the sign bit for negative temperature
                }
                return 0;  // Return 0 for success
            }
        }
    }
    return 1;  // Return 1 for failure
}
```

```c
void gsm_send_sms(const char* phoneNumber, const char* message) {
    // Enable SMS mode
    printf("AT+CMGF=1\r\n");
    __delay_ms(100);

    // Set phone number
    printf("AT+CMGS=\"%s\"\r\n", phoneNumber);
    __delay_ms(100);

    // Set SMS message
    printf("%s", message);
    __delay_ms(100);

    // Send Ctrl+Z to indicate the end of the message
    printf("%c", 0x1A);
    __delay_ms(100);
}

void gsm_cmd(unsigned char *string)
{
    while(*string) {
        tx_data(*string++);
    }
}

void tx_data(unsigned char serial_data)     // Transmit data function
{
    TXREG=serial_data;
    while(PIR1bits.TXIF==0);
}

void putch(char txData)
{
    tx_data(txData);
}
```

## Simulation:

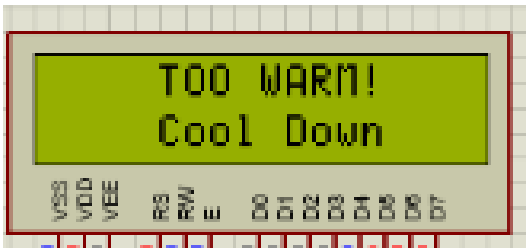


*Figure 3: When Temperature rises.*

*Figure 4: When Humidity rises.*

*Figure 5: When both are high*

## Hardware Connection / Photo of Prototype:

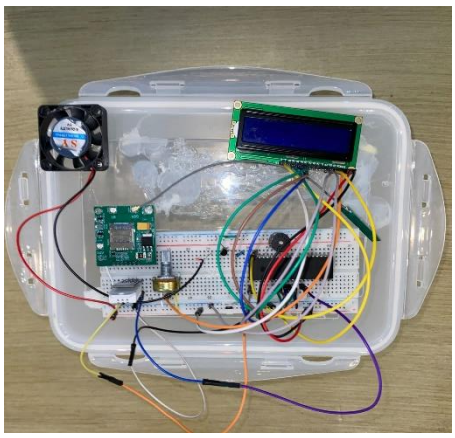

*Figure6:* ***Hardware Connection***

*Figure7:* ***Hardware Connection***

## Conclusion:

In conclusion, the Smart Medicine Storage Box we have designed is the perfect solution to the problem of protecting medicines from heat and humidity. The device is equipped with a display screen for temperature and humidity, as well as sensors that can detect any significant rise in temperature and send a warning message to the user's phone. Our design was based on a thorough literature review, which allowed us to understand the advantages and disadvantages of existing devices, and to develop a product that incorporates the best features while avoiding the drawbacks. We are confident that our Smart Medicine Storage Box will provide an effective and efficient way to store medicines safely and maintain their effectiveness.

## Reference:

- Makres, D. (2022) Hot temperatures can affect your medications, pharmacists say, https://www.wymt.com. Available at: https://www.wymt.com/2022/07/22/hot-temperatures-can-affect-your-medications-pharmacists-say/ (Accessed: 2 Dec 2023).

- Pharmaguideline (no date) Effect of humidity level on tablet stability, Pharmaguideline. Available at: https://www.pharmaguideline.com/2018/09/effect-of-humidity-level-on-tablet-stability.html (Accessed: 2 Dec 2023).

- Personal medicines storage in New Zealand - publish.csiro.au. Available at: https://www.publish.csiro.au/HC/pdf/HC13146 (Accessed: 2 Dec 2023).